Technischer Blueprint und Reverse-Engineering-Spezifikation: KDN.sozial

Umfassende Systemanalyse und Deep-Recherche des Unternehmenskontextes

Die tiefgehende Analyse des Softwareunternehmens KDN.sozial mit Sitz im Technologiepark in Paderborn offenbart einen hochspezialisierten und historisch tief verwurzelten IT-Dienstleister für das kommunale Sozialwesen. KDN.sozial operiert als eigenbetriebsähnliche Einrichtung des KDN (Dachverband kommunaler IT-Dienstleister) in Nordrhein-Westfalen und repräsentiert die Bündelung kommunaler IT-Expertise[1]. Die historischen Wurzeln der Organisation reichen bis in das Jahr 1975 zurück, als das Projekt unter dem Namen SOZ-NW von Kommunen und Rechenzentren als Gemeinschaftsprojekt gegründet wurde[2]. Diese langjährige Historie erklärt die tiefe Durchdringung der Software in den Verwaltungen von Nordrhein-Westfalen, Brandenburg und Sachsen sowie die architektonische Evolution des zugrundeliegenden Technologie-Stacks[2]. Die Kernmission von KDN.sozial besteht in der Bereitstellung ganzheitlicher, medienbruchfreier IT-Lösungen für Jobcenter, Jugendämter und Sozialämter, um die komplexen Anforderungen der Sozialgesetzbücher (insbesondere SGB II, VIII, IX, XII und AsylbLG) digital und rechtssicher abzubilden[4].

Das Produktportfolio ist modular aufgebaut und zielt auf die vollständige Digitalisierung der Sachbearbeitung ab, beginnend beim Onlinezugangsgesetz (OZG) über das Fallmanagement bis hin zur finanziellen Zahlbarmachung. Das Fallmanagement (FMG) ist domänenspezifisch in die Module FMG.job, FMG.jug und FMG.soz unterteilt[5]. Diese Module unterstützen die Fallmanager bei der Analyse des Hilfebedarfs, der Zielsetzung, der Einrichtung von Maßnahmen und der regelmäßigen Evaluation[5]. Die Benutzeroberfläche dieser Module zeichnet sich durch komponentenbasierte Übersichtsmasken, sogenannte Widgets, aus, die eine individuelle Zusammenstellung von Fall- und Personenübersichten ermöglichen[5]. Ergänzt wird das Fallmanagement durch das Leistungsmanagement (LMG), welches das finanzielle Herzstück bildet und die vollautomatisierte Berechnung und Auszahlung von Leistungen übernimmt[8]. Die Architektur ist dabei auf maximale Interoperabilität ausgelegt. Sie integriert Schnittstellen zum Einwohnermeldewesen, zum Ausländerzentralregister, zu Kassenverfahren für Einnahmen und Ausgaben sowie zu standardisierten Übermittlungsprotokollen wie eStatistik.core, GRUSI-DAV und den XÖV-Standards XSozialBasis und XFamilie[5].

Ein weiterer essenzieller Baustein der Produktlandschaft ist das Doku-Tool, welches als hochflexibles Dokumentationssystem konzipiert ist. Es löst starre Erfassungsmasken ab und ermöglicht die Konfiguration individueller Masken mit spezifischen Feldtypen wie Textfeldern, Datumsangaben und Dropdowns[8]. Die Integration in die elektronische Akte (e-Akte) erfolgt über einen DMS-Broker unter Nutzung des CMIS-Standards, was eine direkte Dateiablage und

Massendruckfunktionen direkt aus dem Fachverfahren heraus erlaubt[8]. Ergänzt wird das Portfolio durch spezialisierte Werkzeuge wie den Investitionskosten-Assistenten und die Anbieterdatenbank (ANDA), welche insbesondere für die Heimaufsicht und das Belegungsmanagement von Pflegeeinrichtungen oder Flüchtlingsunterkünften von Bedeutung sind[7].

Der technologische Unterbau dieser massiv verteilten und hochkomplexen Fachverfahren basiert auf einem klassischen, robusten Enterprise-Java-Stack, der für die Anforderungen von Behörden und Rechenzentren optimiert ist. Die Anwendungen sind als Drei-Schichten-Webanwendungen konzipiert, die in Web-Containern wie Apache Tomcat oder IBM WebSphere auf Linux-Servern betrieben werden[10]. Die Basis bildet Java (JDK 11 LTS), was die Präferenz für Langzeitunterstützung und Stabilität im öffentlichen Dienst unterstreicht[10]. Die Präsentationsschicht greift historisch auf JavaServer Pages (JSP) und Struts 1/2 zurück, wobei moderne Überarbeitungen auf Spring MVC und clientseitiges JavaScript (jQuery) setzen, um asynchrone DOM-Manipulationen (AJAX) ohne vollständige Seiten-Reloads zu ermöglichen[10]. Die Persistenzschicht wird durch Hibernate als Object-Relational Mapping (ORM) Tool realisiert, welches mit relationalen Datenbanken wie Oracle oder MySQL kommuniziert[10]. Ein zentrales technisches Merkmal ist die strikte Transaktionssteuerung, die im Legacy-System durch deklarative Frameworks wie Spring @Transactional oder serverseitige JTA-Transaktionsmanager abgebildet wird, um das "Alles-oder-nichts"-Prinzip bei Geldbewegungen zu garantieren[10].

Produktauswahl für den KI-gestützten Nachbau

Basierend auf der tiefgehenden Informationsrecherche fällt die Entscheidung für den Nachbau auf das in sich geschlossene Modul des **Leistungsmanagements (LMG) für die Grundsicherung im Alter und bei Erwerbsminderung nach dem 4. Kapitel SGB XII**.

Diese Entscheidung stützt sich auf mehrere zwingende Kriterien, die für ein erfolgreiches Reverse Engineering und die Programmierung durch einen KI-Agenten essenziell sind. Erstens unterliegt die Geschäftslogik der Grundsicherung strikten, bundesgesetzlich normierten mathematischen Regeln, die öffentlich transparent und exakt dokumentiert sind. Die Berechnung von Regelbedarfsstufen, Mehrbedarfen, Kosten der Unterkunft und die komplexe Einkommensanrechnung nach § 82 SGB XII lassen sich deterministisch in Algorithmen überführen, ohne auf proprietäres Insiderwissen angewiesen zu sein[12]. Zweitens erfordert gerade dieses Modul die Implementierung hochgradig kritischer Transaktionsprozesse. Die Auszahlung von Sozialleistungen verlangt eine absolut fehlerfreie Datenbanktransaktion, bei der Statusänderungen, Kontostandsaktualisierungen und Revisionsprotokolle in einem unteilbaren Schritt (ACID-Prinzip) verarbeitet werden müssen. Dies bietet die ideale Plattform, um den in den Restriktionen geforderten, selbst entwickelten Custom-Transaction-Manager in Java unter Beweis zu stellen[10]. Drittens demonstriert dieses Modul exemplarisch die Notwendigkeit von Concurrency-Control-Mechanismen wie optimistischem und pessimistischem Locking, um "Lost Updates" bei gleichzeitiger Fallbearbeitung durch mehrere Sachbearbeiter zu verhindern[10].

Detaillierte System- und UI-Spezifikation (Reverse

Engineering)

Um das Leistungsmanagement-Modul (LMG) nachzubauen, muss eine detaillierte Spezifikation der Weboberflächen, der zugrundeliegenden mathematischen und fachlichen Prozesse sowie des relationalen Datenmodells erfolgen. Diese Spezifikation dient als direkter Blueprint für die spätere Programmierung.

Spezifikation der Weboberflächen und Dateneingaben

Das Frontend wird als moderne Webanwendung konzipiert, die sich optisch und funktional an die widgetbasierten Dashboards von KDN.sozial anlehnt[5]. Die Architektur des Frontends nutzt HTML5, CSS3 und JavaScript, wobei die Kommunikation mit dem Java-Backend asynchron über RESTful-Endpoints erfolgt. Die Anwendung gliedert sich in vier zentrale Masken.

Die Fallübersicht fungiert als zentrales Dashboard für den Sachbearbeiter. Die Navigation erfolgt über eine linke Seitenleiste, die Zugriff auf Stammdaten, Bedarfsermittlung, Einkommensbereinigung und den Zahlungsverkehr bietet. Der Hauptbereich besteht aus konfigurierbaren Widgets, die den aktuellen Bearbeitungsstatus einer Bedarfsgemeinschaft, offene Aufgaben, Fristen (Wiedervorlagefunktion) und die letzten Einträge im Buchungsjournal anzeigen[5]. Über ein zentrales Suchfeld können Aktenzeichen oder Personennamen per inkrementeller Suche lokalisiert werden.

Die Maske zur Bedarfsermittlung (SGB XII) dient der Erfassung aller Parameter, die für die Berechnung des Gesamtbedarfs relevant sind. Die Eingabefelder sind streng typisiert und mit Validierungsregeln versehen. Ein Dropdown-Menü ermöglicht die Auswahl der Regelbedarfsstufe (RBS), wobei für das Referenzjahr 2026 die RBS 1 (Alleinstehende) mit 563 Euro und die RBS 2 (Partner im gemeinsamen Haushalt) mit 506 Euro hinterlegt sind[12]. Über eine Reihe von Checkboxen können gesetzliche Mehrbedarfe nach § 30 SGB XII aktiviert werden. Wird beispielsweise die Checkbox für "Merkzeichen G / Volle Erwerbsminderung" angewählt, berechnet das System dynamisch im Hintergrund einen Aufschlag von 17 Prozent auf den maßgeblichen Regelbedarf (bei RBS 1 exakt 95,71 Euro)[12]. Ein weiterer Bereich dieser Maske widmet sich den Kosten der Unterkunft und Heizung (KdU). Hier stehen numerische Eingabefelder für die Kaltmiete, die kalten Betriebskosten und die Heizkosten zur Verfügung, welche auf zwei Nachkommastellen validiert werden und in der Summe die Bruttowarmmiete ergeben[13].

Die Maske zur Einkommensbereinigung erfasst alle Zuflüsse, die den Hilfebedarf mindern. Dies umfasst ein Feld für die gesetzliche Alters- oder Erwerbsminderungsrente, welche in der Regel voll anrechenbar ist[13]. Komplexe Logik erfordert das Feld für Erwerbseinkommen. Hier greifen die Freibetragsregelungen des § 82 Abs. 3 SGB XII. Die UI berechnet bereits bei der Eingabe asynchron den Freibetrag, der 30 Prozent des Brutto-Erwerbseinkommens entspricht, jedoch durch eine harte gesetzliche Obergrenze von 50 Prozent der Regelbedarfsstufe 1 (also 281,50 Euro im Jahr 2026) gedeckelt ist[13].

Die finale Maske dient der Zahlbarmachung und der Journalisierung. Sie präsentiert eine tabellarische Historie aller bisherigen Auszahlungen der angewählten Bedarfsgemeinschaft. Ein dedizierter Bereich zeigt den final berechneten Auszahlungsbetrag (Gesamtbedarf abzüglich anrechenbarem Einkommen) als Read-Only-Feld an[13]. Darunter befinden sich Eingabefelder für

die Bankverbindung (IBAN/BIC) des Empfängers. Der zentrale Workflow-Trigger ist der Button "Auszahlung anordnen". Ein Klick auf diesen Button deaktiviert das Element sofort, um versehentliche Doppelbuchungen durch "Double-Clicks" zu verhindern, und sendet einen asynchronen POST-Request an das Backend[9].

Definition der fachlichen Prozesse und Workflows

Der End-to-End-Prozess für die Leistungsberechnung und Auszahlung folgt einem strengen mathematischen und transaktionalen Ablauf, der tief in der Geschäftslogik des Java-Backends verankert ist.

Der Prozess beginnt mit der deterministischen Bedarfsberechnung. Die Service-Schicht summiert den gewählten Regelbedarf, addiert die berechneten prozentualen Mehrbedarfe (z.B. 17% für das Merkzeichen G oder 2,3% für dezentrale Warmwassererzeugung) und fügt die anerkannten Kosten der Unterkunft hinzu. Das Resultat dieser Addition bildet den Gesamtbedarf[12]. Im zweiten Schritt erfolgt die Einkommensbereinigung. Das System prüft das gemeldete Erwerbseinkommen und wendet die 30-Prozent-Regel an. Übersteigt dieser rechnerische Wert die Kappungsgrenze von 281,50 Euro, wird der Freibetrag auf genau diesen Maximalwert fixiert[18]. Das anrechenbare Einkommen ergibt sich somit aus der Summe aller Renten plus dem Erwerbseinkommen abzüglich des berechneten Freibetrags. Der finale Auszahlungsbetrag ist die Differenz aus dem Gesamtbedarf und dem anrechenbaren Einkommen. Ist dieser Wert kleiner oder gleich null, wird der Prozess mit einem fachlichen Hinweis abgebrochen[13].

Der kritischste Prozess ist die transaktionale Zahlbarmachung. Sobald der Request vom Server empfangen wird, extrahiert das System eine vom Frontend generierte, eindeutige Vorgangsnummer. Diese dient der Idempotenz: Das System prüft zunächst in der Datenbank, ob diese Vorgangsnummer bereits existiert, um Doppelbuchungen bei Netzwerkausfällen oder wiederholten Requests auszuschließen[10]. Anschließend öffnet das System eine Datenbanktransaktion. Um "Lost Updates" durch parallele Sachbearbeiter zu verhindern, wird der Datensatz der betroffenen Bedarfsgemeinschaft mit einem pessimistischen Lock (SELECT ... FOR UPDATE) gesperrt[10]. Das System verifiziert, ob der berechnete Zahlbetrag das verfügbare Restbudget der Bedarfsgemeinschaft nicht übersteigt. Ist die Deckung gegeben, werden drei Datenbankoperationen innerhalb derselben Transaktion ausgeführt: Die Anlage des Zahlungsdatensatzes mit dem Status 'BEREIT', die Minderung des Restanspruchs auf dem Kontostand der Bedarfsgemeinschaft und das Schreiben eines revisionssicheren Audit-Eintrags in das Buchungsjournal[10]. Erst wenn alle drei Operationen erfolgreich in das Redo-Log der Datenbank geschrieben wurden, wird die Transaktion durch ein commit() bestätigt und der Status HTTP 200 an das Frontend gemeldet[9].

Ableitung des relationalen Datenbankmodells

Aus den fachlichen und UI-spezifischen Anforderungen wird ein relationales Datenmodell in der dritten Normalform (3NF) abgeleitet, welches spezifisch auf eine Oracle-Datenbank zugeschnitten ist. Das Modell nutzt Oracle Sequences zur Generierung künstlicher Primärschlüssel und implementiert Mechanismen für optimistisches Locking.

Tabellenname	Spaltenname	Oracle Datentyp	Constraints und fachliche Bedeutung
BEDARFSGEMEINSCHAFT	ID	NUMBER(19)	PRIMARY KEY, generiert über SEQ_BG.NEXTVAL
	AKTENZEICHEN	VARCHAR2(50)	UNIQUE NOT NULL, Fachlicher Identifikator für die Behörde
	HAUPTPERSON_NAME	VARCHAR2(100)	NOT NULL, Name des Antragstellers
	STATUS_AKTIV	NUMBER(1)	DEFAULT 1, Kennzeichen für aktive Fälle (1 = aktiv, 0 = historisiert)
	BUDGET_REST	NUMBER(10,2)	Verfügbares Restbudget oder Maximalanspruch pro Monat
	VERSION	NUMBER(10)	DEFAULT 1, Spalte für Optimistic Locking, wird durch einen Before-Update-Trigger inkrementiert[15]
LEISTUNGS_FALL	ID	NUMBER(19)	PRIMARY KEY, generiert über SEQ_FALL.NEXTVAL
	BG_ID	NUMBER(19)	FOREIGN KEY

			referenziert BEDARFSGEMEINSCHAFT(ID)
	REGELBEDARFSSTUFE	NUMBER(1)	Erlaubte Werte: 1 bis 6, steuert den Basisbetrag[12]
	FLAG_MERKZEICHEN_G	NUMBER(1)	DEFAULT 0, Indikator für den 17% Mehrbedarf (1 = Ja)[16]
	KDU_MIETE	NUMBER(10,2)	Anerkannte Kaltmiete in Euro[13]
	KDU_HEIZUNG	NUMBER(10,2)	Anerkannte Heizkosten in Euro[13]
	EINKOMMEN_RENTE	NUMBER(10,2)	Voll anrechenbare Renteneinkünfte[13]
	EINKOMMEN_ERWERB	NUMBER(10,2)	Brutto-Erwerbseinkommen zur Berechnung des Freibetrags[18]
ZAHLUNG	ID	NUMBER(19)	PRIMARY KEY, generiert über SEQ_ZAHLUNG.NEXTVAL
	BG_ID	NUMBER(19)	FOREIGN KEY referenziert BEDARFSGEMEINSCHAFT(ID)
	VORGANGS_NR	VARCHAR2(100)	UNIQUE NOT NULL, UUID zur Sicherstellung der

			Idempotenz und Verhinderung von Doppelbuchungen[10]
	ZAHLUNGSMONAT	VARCHAR2(7)	NOT NULL, Abrechnungszeitraum im Format YYYY-MM
	AUSZAHLUNGSBETRAG	NUMBER(10,2)	NOT NULL, Der final berechnete und angewiesene Betrag
	ZAHLUNGSSTATUS	VARCHAR2(20)	NOT NULL, Erlaubte Werte: 'BEREIT', 'ÜBERGEBEN', 'STORNIERT'. Gesteuert über Statusflags für Batchläufe[10]
JOURNAL	ID	NUMBER(19)	PRIMARY KEY, generiert über SEQ_JOURNAL.NEXTVAL
	BG_ID	NUMBER(19)	Logische Referenz zur BG. Ein harter Foreign Key wird oft vermieden, um Historisierung auch bei gelöschten BGs zu wahren
	AKTION_TEXT	VARCHAR2(500)	NOT NULL, Beschreibender Text, z.B. "Auszahlung über 450,00 EUR für

			2026-09 angeordnet"
	SACHBEARBEITER_ID	VARCHAR2(50)	NOT NULL, Identifikator des ausführenden Benutzers zur Erfüllung des Vier-Augen-Prinzips und der DSGVO[10]
	ERSTELLT_AM	TIMESTAMP	DEFAULT SYSTIMESTAMP, Exakter Zeitpunkt der Transaktion

Technisches Architekturkonzept und Lösungsdesign

Die technischen Vorgaben für den Nachbau fordern eine signifikante architektonische Herausforderung: Die Anwendung muss ein reines Java-Backend für eine Oracle-Datenbank bereitstellen, wobei für das Transaktionsmanagement explizit der Einsatz fertiger Frameworks wie Spring @Transactional oder Java Transaction API (JTA) untersagt ist. Dies erzwingt die Programmierung eines proprietären Custom-Transaction-Managers, der die ACID-Eigenschaften auf tiefster Ebene sicherstellt. Das Konzept integriert dabei sinnvolle Elemente des originalen KDN.sozial Tech-Stacks, wie Apache Tomcat, reine Servlets, JSP/jQuery-Konzepte für das Frontend und ein robustes Dependency-Management[10].

Datenbankverbindungs-Management (Oracle UCP)

Die Basis für performante Datenbankoperationen bildet das Connection Pooling. Anstelle generischer Pools wie HikariCP wird hier der Oracle Universal Connection Pool (UCP) eingesetzt. UCP (ucp.jar) ist hochgradig für Oracle-Datenbanken optimiert und bietet native Unterstützung für Fast Connection Failover (FCF) sowie Load Balancing in Real Application Clusters (RAC) Umgebungen, was im hochverfügbaren Behördenumfeld essenziell ist[24].
Die Initialisierung des Pools erfolgt einmalig beim Start der Applikation (beispielsweise über einen ServletContextListener im Tomcat). Die Konfiguration nutzt die Klasse oracle.ucp.jdbc.PoolDataSourceFactory, um eine PoolDataSource zu instanziieren. Wichtige Parameter wie InitialPoolSize, MinPoolSize und MaxPoolSize werden definiert, um das Verhalten des Pools unter Last zu steuern. Ebenso wird die ConnectionFactoryClassName auf die native Implementierung oracle.jdbc.datasource.impl.OracleDataSource gesetzt, um die modernsten Treiberfunktionen zu nutzen[25]. Verbindungen werden bei Bedarf über die Methode getConnection() aus dem Pool entliehen und nach der Transaktion mit close() an den Pool zurückgegeben, ohne physisch abgebaut zu werden[28].

Architektur des Custom-Transaction-Managers

Das Herzstück des Architekturkonzepts ist der Custom-Transaction-Manager, der deklaratives Transaktionsmanagement in reinem Java nachbildet, ähnlich der Mechanik, die Spring unter der Haube verwendet. Dieses Design basiert auf drei zentralen Komponenten: Einem Thread-lokalen Speicher, einer benutzerdefinierten Annotation und einem dynamischen Proxy mit einem Invocation Handler.
Da JDBC-Transaktionen stets an ein einzelnes java.sql.Connection-Objekt gebunden sind, müssen alle Datenbankzugriffe (Data Access Objects, DAOs), die innerhalb eines Requests stattfinden, exakt dieselbe Verbindung nutzen. Um dies ohne Parameterübergabe durch alle Methodensignaturen zu erreichen, wird ein ThreadLocal<Connection> eingesetzt[11]. Der ThreadLocal-Speicher garantiert, dass jeder HTTP-Request (der in einem eigenen Tomcat-Thread läuft) einen isolierten Zugriff auf seine spezifische Datenbankverbindung hat.
Ein kritisches Detail hierbei ist das Memory-Leak-Management: Nach Abschluss der Transaktion muss die Verbindung zwingend durch den Aufruf von remove() aus dem ThreadLocal gelöscht werden, da Thread-Pools in Web-Containern Threads wiederverwenden und andernfalls "schmutzige" Zustände in neue Requests leaken würden[29].
Um die Transaktionsgrenzen im Code elegant zu definieren, wird eine Meta-Annotation @CustomTransactional geschaffen. Service-Methoden, die fachliche Einheiten bündeln (wie die Methode zur Auszahlung), werden mit dieser Annotation markiert. Das Framework muss nun diese Methoden abfangen. Da Spring-AOP verboten ist, wird das native Java Proxy Pattern (java.lang.reflect.Proxy) genutzt. Beim Bootstrapping der Applikation werden die Service-Klassen nicht direkt instanziiert, sondern durch eine Proxy-Factory erzeugt, die das originale Interface implementiert und einen InvocationHandler injiziert[31].
Der Ablauf im InvocationHandler ist streng sequenziell:

1. Wird eine Methode aufgerufen, analysiert der Handler per Reflection, ob die @CustomTransactional Annotation vorliegt.
2. Ist dies der Fall, wird eine Connection aus der Oracle UCP-PoolDataSource entliehen.
3. Der Autocommit-Modus wird deaktiviert (connection.setAutoCommit(false)), womit die Transaktion offiziell beginnt[29].
4. Die Connection wird in den ThreadLocal-Kontext gelegt, sodass nachgelagerte DAO-Aufrufe sie abrufen können.
5. Die eigentliche Geschäftslogik wird via method.invoke(target, args) ausgeführt.
6. Das Ergebnis wird abgefangen. Schließt die Methode ohne Fehler ab, führt der Handler ein connection.commit() aus. Die Daten werden im Oracle Redo-Log auf die Festplatte geschrieben und dauerhaft[10].
7. Wirft die Geschäftslogik eine Exception (die vom Proxy als InvocationTargetException gekapselt wird), fängt der Handler diese im catch-Block ab. Er entscheidet anhand des Fehlertyps, ob ein connection.rollback() ausgeführt wird, um die Atomarität zu wahren[29].
8. Ein unabdingbarer finally-Block stellt sicher, dass die connection.close() aufgerufen wird (Rückgabe in den Pool) und das ThreadLocal bereinigt wird[29].

Concurrency Control: Optimistisches und Pessimistisches Locking

Im Kontext von Sozialdaten muss zwingend verhindert werden, dass sich Transaktionen gegenseitig überschreiben ("Lost Update"). Das Konzept nutzt eine hybride Strategie.

Für den Dialogbetrieb im Web-Frontend wird optimistisches Locking eingesetzt. Die Tabelle BEDARFSGEMEINSCHAFT erhält eine Spalte VERSION. Ein PL/SQL-Trigger in der Oracle-Datenbank (BEFORE UPDATE) stellt sicher, dass diese Version bei jeder Änderung inkrementiert wird, oder die Java-Schicht verwaltet die Iteration manuell[14]. Wenn ein Sachbearbeiter eine Maske öffnet, wird die aktuelle Version in den Client geladen. Beim Speichern wird geprüft: UPDATE ... WHERE ID = ? AND VERSION = ?. Wurde der Datensatz in der Zwischenzeit durch einen Kollegen verändert, schlägt das Update fehl (es liefert 0 betroffene Zeilen). Die Java-Logik wirft daraufhin eine Exception, der Transaction-Manager rollt zurück, und der Nutzer erhält die Warnung "Datensatz wurde zwischenzeitlich geändert"[10].
Für den eigentlichen, kurzen Buchungsprozess in der Datenbank wird pessimistisches Locking verwendet. Die DAO-Methode lädt den zu bebuchenden Datensatz mit dem Zusatz SELECT ... FOR UPDATE[10]. Dies platziert eine exklusive Zeilensperre in der Oracle-Datenbank. Parallele Threads, die denselben Datensatz ansteuern, müssen auf Datenbankebene warten, bis der erste Thread ein Commit oder Rollback ausführt[10].
Kommt es durch komplexe Transaktionsgeflechte zu einem Kreuz-Warten zweier Threads, erkennt Oracle dies als Deadlock und bricht eine der Transaktionen mit dem Fehler ORA-00060: deadlock detected ab[33]. Der Custom-Transaction-Manager ist so konzipiert, dass er im catch-Block den spezifischen Oracle Error Code 60 abfängt. Anstatt den Endnutzer mit einem Systemfehler zu konfrontieren, führt der Handler ein Rollback aus, wartet ein kurzes Intervall und versucht den gesamten Methodenaufruf bis zu dreimal automatisch erneut (Retry-Mechanismus)[10]. Dies garantiert eine extrem hohe Systemresilienz.

Blueprint: Schritt-für-Schritt KI-Programmierung

Dieser Blueprint liefert die exakten Anweisungen, um einen KI-Agenten (z.B. GitHub Copilot oder ein eigenständiges LLM) durch den schrittweisen Aufbau des Demosystems zu steuern. Die Restriktionen müssen dem Agenten vor jedem Schritt klargemacht werden.

Phase 1: Datenbank-Initialisierung (Oracle SQL DDL)

Weise die KI an, das relationale Datenmodell als DDL-Skript für eine Oracle-Datenbank zu generieren.

- **Vorgabe:** Erstellung der Tabellen BEDARFSGEMEINSCHAFT, LEISTUNGS_FALL, ZAHLUNG und JOURNAL gemäß der Spezifikation in Abschnitt 3.
- **Schlüssel:** Für jede Tabelle muss eine Oracle SEQUENCE (z.B. SEQ_BG START WITH 1 INCREMENT BY 1) erstellt werden, da Oracle klassisches Auto-Increment anders handhabt.
- **Trigger-Logik:** Fordere die Generierung eines PL/SQL BEFORE UPDATE Triggers für die Tabelle BEDARFSGEMEINSCHAFT an. Der Trigger muss prüfen, ob die VERSION von außen hochgezählt wurde, und falls nicht, die :new.VERSION auf :old.VERSION + 1 setzen, um die Basis für das optimistische Locking zu schaffen[14].

Phase 2: Infrastruktur und Connection Pooling

Weise die KI an, das Verbindungsmanagement aufzubauen.

- **Vorgabe:** Integration von ucp.jar und ojdbc8.jar.
- **Klasse DatabaseConnectionManager:** Erstellung einer Singleton-Klasse. Innerhalb der

Initialisierung soll die Factory-Methode oracle.ucp.jdbc.PoolDataSourceFactory.getPoolDataSource() aufgerufen werden. Die KI muss die Setter für URL, User, Password, InitialPoolSize und ConnectionFactoryClassName (oracle.jdbc.datasource.impl.OracleDataSource) implementieren[25]. Die Klasse muss eine Methode DataSource getDataSource() bereitstellen.

Phase 3: Entwicklung des Custom-Transaction-Managers

Dies ist die komplexeste Phase. Führe die KI präzise durch die Erstellung der AOP-Alternative.

- **Vorgabe 1:** Definition einer simplen @interface CustomTransactional mit Target METHOD.
- **Vorgabe 2:** Erstellung der Klasse TransactionContext. Diese benötigt ein private static final ThreadLocal<Connection> CONNECTION_HOLDER = new ThreadLocal<>();. Füge statische Methoden bind(Connection), getConnection() und clear() hinzu. Die Methode getConnection() muss eine Exception werfen, falls keine Connection gebunden ist (Fail-Fast).
- **Vorgabe 3:** Programmierung der Klasse TransactionProxyHandler implementierend java.lang.reflect.InvocationHandler[31].
- **Logik-Implementierung für die KI:** Die Methode invoke(Object proxy, Method method, Object[] args) muss prüfen, ob die Annotation vorliegt. Falls ja:

1. Connection über den DatabaseConnectionManager beziehen.
2. connection.setAutoCommit(false) setzen.
3. TransactionContext.bind(connection) aufrufen.
4. In einem try-Block method.invoke(target, args) ausführen und danach connection.commit() aufrufen.
5. Im catch (InvocationTargetException e)-Block die Fehlerursache (e.getCause()) analysieren. Handelt es sich um eine SQLException mit dem Error Code 60 (ORA-00060), implementiere eine For-Schleife für 3 Retries (inklusive Thread.sleep und Rollback vor dem Retry)[33]. Bei anderen Fehlern ein hartes connection.rollback() erzwingen und die Exception weiterwerfen[29].
6. Im finally-Block zwingend TransactionContext.clear() und connection.close() ausführen, um Memory Leaks zu verhindern[29].

- **Vorgabe 4:** Eine Factory-Klasse ServiceFactory, die Instanzen via Proxy.newProxyInstance(classLoader, interfaces, new TransactionProxyHandler(target)) erzeugt.

Phase 4: Data Access Objects (DAOs) und SQL-Sicherheit

Weise die KI an, die Persistenzschicht zu schreiben.

- **Vorgabe:** Erstellung von DAO-Klassen wie BedarfsgemeinschaftDao und ZahlungDao.
- **Striktion:** DAOs dürfen keine eigenen Verbindungen öffnen oder schließen. Jede DAO-Methode muss ihre Connection über TransactionContext.getConnection() beziehen.
- **Sicherheit:** Sämtliche SQL-Abfragen müssen zwingend über PreparedStatement abgewickelt werden, um SQL-Injection auszuschließen[10].
- **Pessimistisches Locking:** Das BedarfsgemeinschaftDao muss eine Methode

lockForUpdate(long bgId) implementieren, die den SQL-Befehl SELECT * FROM BEDARFSGEMEINSCHAFT WHERE ID = ? FOR UPDATE absetzt[10].

Phase 5: Geschäftslogik (SGB XII) und Service-Schicht

Weise die KI an, die fachliche Berechnung zu implementieren.

- **Vorgabe:** Erstellung eines Interfaces LeistungsService und dessen Implementierung LeistungsServiceImpl.
- **Berechnungslogik:** Implementierung der SGB XII Regeln. Die Methode berechnet den Gesamtbedarf (Regelbedarf + Mehrbedarf 17% + Miete + Heizung). Für das Einkommen wird der Freibetrag berechnet: Math.min((bruttoErwerb * 0.30), 281.50)[13]. Das anrechenbare Einkommen wird vom Bedarf abgezogen.
- **Auszahlungslogik:** Die Methode buchungAuszahlung(long fallId, String vorgangsNr) wird mit @CustomTransactional annotiert.
- **Ablauf-Implementierung für die KI:**

1. Prüfe via DAO, ob die vorgangsNr in der Tabelle ZAHLUNG existiert. Wenn ja, beende den Prozess (Idempotenz)[10].
2. Führe lockForUpdate(bgId) aus.
3. Lade die aktuellen Daten, berechne den Zahlbetrag. Ist dieser <= 0, wirf eine FachlicheException (führt zum Rollback).
4. Führe ZahlungDao.insert() aus.
5. Führe JournalDao.insert() aus, um den Sachbearbeiter und die Aktion zu protokollieren (Revisionssicherheit)[10].

Phase 6: Web-Frontend und REST-Integration

Weise die KI an, die Kommunikationsschicht zu programmieren.

- **Vorgabe Backend:** Schreibe native Java HttpServlet-Klassen, die den eingebetteten Tomcat-Server nutzen (Brücke zum originalen Tech-Stack[10]). In der init()-Methode des Servlets wird der Service über die ServiceFactory als Proxy geladen. Der Servlet nimmt POST-Requests (JSON) unter /api/auszahlung entgegen, deserialisiert diese (z.B. mit der Jackson-Bibliothek), ruft den geproxten Service auf und meldet HTTP 200 oder HTTP 400 zurück.
- **Vorgabe Frontend:** Erstelle eine Single-Page-Application (HTML/CSS), die das Widget-Design von KDN.sozial simuliert[5]. Nutze JavaScript (oder explizit jQuery für die Legacy-Brücke[10]), um Formulardaten per AJAX an den /api/auszahlung Endpunkt zu senden. Das Frontend muss Fehler abfangen und bei HTTP 400 (insbesondere bei Warnungen zum optimistischen Locking) klare Dialoge für den Sachbearbeiter rendern, während bei HTTP 200 die Tabelle der historischen Auszahlungen asynchron durch DOM-Manipulation aktualisiert wird.

Referenzen

1. Über den KDN - KDN Dachverband, https://www.kdn.de/ueber-den-kdn/
2. KDN.sozial — Firma im DACH B2B-Softwaremarkt | The Playbook, https://www.the-playbook.de/de/firmen/kdn-sozial/

3. In eigener Sache - KDN.sozial, https://kdn-sozial.de/in-eigener-sache/
4. KDN.sozial | IT-Lösungen für Jobcenter, Jugend- und Sozialämter, https://kdn-sozial.de/
5. Software-Lösungen für Jugendämter | SGB VIII - KDN.sozial, https://kdn-sozial.de/leistungen/jugendaemter/
6. Software-Lösungen für Jobcenter | Fallmanagement - KDN.sozial, https://kdn-sozial.de/leistungen/jobcenter/
7. Software-Lösungen für Sozialämter | SGB XII und AsylbIG - KDN.sozial, https://kdn-sozial.de/leistungen/sozialaemter/
8. Das neue DOKU-TOOL - KDN.sozial, https://kdn-sozial.de/doku-tool/
9. Software-Lösungen für Behörden und Kommunen - KDN.sozial, https://kdn-sozial.de/digitalisierung/
10. Tech_Stack_Vorbereitung_KDN.sozial.pdf
11. Handling transactions across multiple data sources in Spring Boot, https://atlantbh.com/blog/handling-transactions-across-multiple-data-sources-in-spring-boot/
12. Grundsicherung im Alter und bei Erwerbsminderung (SGB XII, 4, https://sozialrat.org/grundsicherung-alter-erwerbsminderung/
13. Grundsicherung im Alter-Rechner 2026 – Höhe & Anspruch, https://www.deutschland-rechner.de/grundsicherung-im-alter-rechner
14. Optimistic Locking with Concurrency in Oracle - OraFAQ, https://www.orafaq.com/papers/locking.pdf
15. Handle CONCURRENT Updates(Pessimistic and Optimistic Oracle, https://tipsfororacle.blogspot.com/2017/04/handle-concurrent-updatespessimistic.html
16. Grundsicherung Höhe 2026: Sätze, Tabelle, Rechenweg, https://www.juraforum.de/lexikon/grundsicherung-hoehe
17. Grundsicherung-Rechner 2026: Anspruch im Alter berechnen, https://rechner-portal.de/finanzen/sozialleistungen/grundsicherung-rechner
18. Hinzuverdienst bei Grundsicherung im Alter - JuraForum.de, https://www.juraforum.de/lexikon/hinzuverdienst-bei-grundsicherung-im-alter
19. Rente: Sozialämter lassen bis zu 281,50 Euro Freibetrag monatlich, https://www.gegen-hartz.de/news/rente-sozialaemter-lassen-bis-zu-28150-euro-freibetrag-monatlich-bei-grundsicherung-verfallen
20. Software für Behörden und Kommunen | Termine und Messen, https://kdn-sozial.de/termine/
21. WIll Select for update wait locks the rows permanently?, https://stackoverflow.com/questions/33373352/will-select-for-update-wait-locks-the-rows-permanently
22. Implementing Optimistic Locking in Oracle - java - Stack Overflow, https://stackoverflow.com/questions/41006941/implementing-optimistic-locking-in-oracle
23. Anwendertreffen 2023 - KDN.sozial, https://kdn-sozial.de/anwendertreffen-2023/
24. Oracle Connection Pooling in Spring Boot: When to Use HikariCP, https://redstack.dev/2026/06/11/oracle-connection-pooling-in-spring-boot-when-to-

use-hikaricp-and-when-to-use-ucp/
25. UCP Best Practices for Oracle Database and Spring Boot | developers, https://blogs.oracle.com/developers/ucp-best-practices-for-oracle-database-19c-and-spring-boot
26. UCP Frequently Asked Questions (FAQ) - Oracle, https://www.oracle.com/database/technologies/universal-connection-pool-faq.html
27. oracle.jdbc.datasource (Oracle Database JDBC Java API Reference), https://docs.oracle.com/en/database/oracle/oracle-database/21/jajdb/oracle/jdbc/datasource/package-summary.html
28. High-Performance Oracle JDBC Programming, https://www.oracle.com/technical-resources/articles/vasiliev-oracle-jdbc.html
29. Spring Boot @Transactional: AOP, Propagation & Pitfalls - Medium, https://medium.com/@nelushgayashan/a-deep-dive-into-spring-boots-transactional-494377e70a66
30. alltypes - spring-framework, https://docs.spring.io/spring-framework/docs/5.0.2.RELEASE/kdoc-api/spring-framework/alltypes/index.html
31. The Hidden Power of Java Proxies: JDK and CGLIB Made Simple, https://fuad-ahmadov.medium.com/the-hidden-power-of-java-proxies-jdk-and-cglib-made-simple-7736ba08b5f8
32. What are some forbidden, broken, possibly even black magic stuff, https://www.reddit.com/r/java/comments/qpepoe/what_are_some_forbidden_broken_possibly_even/
33. ORA-00060 | DEADLOCK DETECTED, https://dba.stackexchange.com/questions/145889/ora-00060-deadlock-detected
34. 8 SQL Processing for Application Developers - Oracle Help Center, https://docs.oracle.com/en/database/oracle/oracle-database/19/adfns/sql-processing-for-application-developers.html